

Demo runbook — two laptops

Follow this in order. Budget ~25 minutes for setup, then rehearse once.

Before you start

On **both** laptops:

```

bash
python3 --version    # must be 3.8+
```

Find Laptop B's LAN IP:

```

bash
# Linux / macOS
ip addr | grep "inet " | grep -v 127.0.0.1  # or: ifconfig | grep "inet "
# Windows
ipconfig
```

Test that A can reach B at all:

```

bash
ping <B-ip>
```

If ping fails, nothing else will work. Fix the network first — same Wi-Fi, client isolation off, or use a phone hotspot / an ethernet cable.

Step 0 — Cluster certificate (once, before anything else)

On one laptop:

```

bash
cd memcloud
python3 -m memcloud.security --out cluster
```

Copy `cluster.pem` **and** `cluster.key` to the other laptop, into the same folder. Without them the second node cannot connect at all.

Step 1 — Start Laptop B (lends RAM)

```

bash
cd memcloud
python3 -m memcloud.node --name laptop-b --reserve-mb 4096 --budget-mb 64
```

Check the banner says `(TCP + mutual TLS)` and note the fingerprint.

Confirm the banner shows `data plane <B-ip>:47801`. If it shows `127.0.0.1`, B is not on the network.

The fingerprint on both laptops must be **identical**. If it is not, you copied only one of the two files, or one node generated its own.

Firewall: if A cannot connect, allow inbound TCP 47801 on B.

```

bash
# Linux (ufw)
sudo ufw allow 47801/tcp
# macOS: System Settings > Network > Firewall > allow python3
# Windows: allow python.exe on Private networks when prompted
```

Step 2 — Start Laptop A (runs the app)

```

bash
cd memcloud
python3 -m memcloud.node --name laptop-a --peer <B-ip>:47801 --budget-mb 512
```

Step 3 — Confirm the link before demoing

Open <http://127.0.0.1:5892> on Laptop A.

You must see **two** node cards, both with a green dot, and a non-zero round-trip figure on laptop-b. If B shows offline, go back to Step 1.

Quick check from the terminal:

```

bash
curl -s http://127.0.0.1:5892/api/nodes | python3 -m json.tool | head -30
```

Step 4 — The demo itself

Beat 1: the problem

Point at Laptop A's card. Local budget is 512 MB. Point at Laptop B's card: several GB of RAM sitting idle, doing nothing.

Beat 2: fill local RAM

Set frames to **600** and press **Seed cache**.

Watch the local RAM figure climb to the 512 MB budget and stop. That is the overflow point.

Beat 3: overflow into the peer

Seeding continues past the budget. On B's card, **hosting for peers** starts rising, and so does B's process RSS. Those bytes are physically resident in the other laptop's RAM right now.

Prove it from B's own terminal if you want:

```

bash
curl -s http://127.0.0.1:5892/api/cache | python3 -c \
"import json,sys;d=json.load(sys.stdin)['memcloud'];print('local',d['local_bytes'],'remote',d['remote_bytes'])"
```

Beat 4: read it back

Press **Run reads** with 200 reads.

The table fills with **LOCAL** and **PEER** rows and their measured latencies. The tick in the last column is a CRC32 check — every frame that came back over the network is byte-identical to what went out.

Beat 5: it is real image data

The preview pane renders a frame fetched through MemCloud. If the source header says **PEER**, those pixels came off the other laptop.

Show it directly:

```

bash
curl -sI http://127.0.0.1:5892/api/frame/frame_00420 | grep MemCloud
```

Beat 6: memory pressure relief

Press **Spill 25% to peers**.

Laptop A's MemCloud RAM drops. Laptop B's hosted RAM rises by the same amount. Run reads again — the spilled frames now serve from **PEER** and still verify.

Beat 7: security

The header shows an **mTLS** pill. Prove it is enforced — from any machine, try to connect without the certificate:

```
bash
curl -sv telnet://<B-ip>:47801 </dev/null
```

Laptop B's terminal prints `[security] rejected <ip> -- WRONG_VERSION_NUMBER` and the connection dies before a single byte of protocol is exchanged. The rejection also appears in the dashboard event feed.

Every block in this demo crossed the network inside TLS 1.3 with AES-256-GCM.

Beat 8: the disk tier

Press **Reset**, then **Seed cache** with a frame count larger than local budget + B's reserve combined. The excess never enters RAM and serves from `DISK`, which is the slow path MemCloud exists to avoid.

What to say when asked hard questions

"Is this just a cache?" It is a memory tier. The distinguishing property is that the tier lives in another machine's physical RAM, allocated on demand by a placement scheduler, not on local disk.

"What happens if Laptop B dies?" Right now, the blocks it held are lost and reads fall through to the disk cold store. There is no replication yet. That is our next milestone. — *Say this plainly. Do not claim recovery works.*

"Is it faster than disk?" On a LAN, a peer RAM read is a network round trip plus a memory copy. Our disk numbers in this demo are page-cache-warm, so they flatter disk. The honest claim is architectural, not a benchmark win we have measured on cold disk yet.

"Is it secure?" The data plane is mutual TLS 1.3 with AES-256-GCM. Both ends must present the shared cluster certificate, so an unauthorised host cannot read or write blocks — we can demonstrate the rejection live. The honest limitation is that it is *one shared key for the whole cluster*: it authenticates membership, not individual identity, and there is no revocation. Per-node certificates signed by a cluster CA is the next step.

"Can this run an LLM across both laptops' RAM?" Not model weights, and not a live decode-time KV cache — a LAN round trip is 1-25 ms and a token budget at 30 tok/s is ~1 ms per layer, so that is three orders of magnitude off. What does fit is precomputed **prefix** KV caches and RAG corpora, which are bulk sequential transfers. We built the storage side for that. We have not wired it into an inference engine yet. Full arithmetic is in LLM_NOTES.md.

"Did you build on OpenFabric?" We used it as an architectural reference for discovery and cluster telemetry. Its RAM feature is a read-only aggregated view — it has no protocol for placing memory on a peer. The remote allocation layer here is new work.

If something breaks mid-demo

symptom	fix
B shows offline	check firewall on B, re-run A with the right <code>--peer</code>
everything says LOCAL	local budget too big; restart A with a smaller <code>--budget-mb</code>
everything says DISK	B not connected, or B's reserve is exhausted
seed is slow	fewer frames, or smaller: <code>?width=320&height=240</code>
port already in use	pass a different <code>--data-port</code> / <code>--api-port</code>
<code>PEER_DID_NOT_RETURN_A_CERTIFICATE</code>	the other laptop is missing <code>cluster.key</code>
fingerprints differ	you copied only one file; copy both, restart
TLS breaks and time is short	add <code>--insecure</code> on both nodes and say so
Fallback if the network dies completely: run both nodes on one laptop with different ports and <code>--peer 127.0.0.1:47811</code> . The transfer is still real TCP, just over loopback. Say so if you use it.	